

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) ¹
Student Name	Saravana Senthilkumar
Register Number	312224700157
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of ten machine learning classifiers. Each model was trained and validated (i) in the original four-dimensional feature space (No-PCA) and (ii) in a PCA-reduced feature space (With-PCA), with hyperparameter tuning and 5-fold stratified cross-validation performed in both settings so that the two configurations could be fairly compared on accuracy, F1-score, and stability across folds.

2 Problem Statement

Given the Iris flower measurements dataset, the task is to classify each sample into one of three species (*Iris-setosa*, *Iris-versicolor*, *Iris-virginica*) using its four numeric features (sepal length, sepal width, petal length, petal width).

- **Input:** A 150×4 feature matrix of standardized sepal/petal measurements.
- **Output:** A predicted class label (one of the three Iris species) for each sample.
- **Objective:** Compare ten classifiers — SVM, Naïve Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and Stacking — trained on the raw standardized features against the same classifiers trained on PCA-reduced components, and determine whether dimensionality reduction improves or degrades predictive performance and cross-validation stability.

3 Dataset Description

Dataset Name	Iris Flower Dataset
Dataset Source	UCI Machine Learning Repository / <code>Iris.csv</code> (also available as <code>sklearn.datasets.load_iris</code>)
Number of Samples	150
Number of Features	4 (SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm)
Number of Classes	3 (Iris-setosa, Iris-versicolor, Iris-virginica)
Class Distribution	Perfectly balanced — 50 samples per class
Missing Values	None

Train-Test Split	80% train / 20% test (120 train, 30 test), stratified split, <code>random_state=42</code>
------------------	---

4 Exploratory Data Analysis

Class Distribution

Table 2: Class Distribution — Iris Dataset

Species	Sample Count	Proportion
Iris-setosa	50	33.3%
Iris-versicolor	50	33.3%
Iris-virginica	50	33.3%

Inference:

The dataset is perfectly balanced, with exactly 50 samples belonging to each of the three species. This eliminates the need for class-balancing techniques such as oversampling or class-weighting, and it means accuracy is a reliable metric for comparing models since no single class can dominate the score. It also simplifies interpretation of the confusion matrix and cross-validation folds, as each fold is expected to contain roughly equal numbers of each class under stratified sampling.

Correlation Matrix

Table 3: Feature Correlation Matrix

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
SepalLengthCm	1.00	−0.12	0.87	0.82
SepalWidthCm	−0.12	1.00	−0.43	−0.37
PetalLengthCm	0.87	−0.43	1.00	0.96
PetalWidthCm	0.82	−0.37	0.96	1.00

Inference:

Petal length and petal width are very strongly correlated with each other, and both are also strongly correlated with sepal length. Sepal width, in contrast, is only weakly (and negatively) correlated with the other three features. This high degree of redundancy among the petal- and sepal-length-related features is exactly the kind of structure PCA is designed to exploit — it suggests that most of the variance in the dataset can be captured by a small number of orthogonal components, which is later confirmed by the PCA variance table showing 2 components explaining over 95% of the variance.

Statistical Summary

Table 4: Statistical Summary of Iris Features

Statistic	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
Mean	5.84	3.06	3.76	1.20
Std. Dev.	0.83	0.44	1.77	0.76
Min	4.30	2.00	1.00	0.10
25%	5.10	2.80	1.60	0.30
50% (Median)	5.80	3.00	4.35	1.30
75%	6.40	3.30	5.10	1.80
Max	7.90	4.40	6.90	2.50

Inference:

Petal length and petal width show far higher relative spread (large gap between min and max, wide interquartile range) than the sepal measurements, consistent with these two features carrying most of the between-species variation. Sepal width has the smallest standard deviation of all four features, reinforcing that it varies least across the dataset and contributes comparatively little discriminative power on its own.

5 Data Preprocessing

- **Missing value handling:** Not required — the Iris dataset contains no missing values.
- **Encoding:** The target column (`Species`) was label-encoded into integers 0, 1, 2 using `sklearn.preprocessing.LabelEncoder`, which is required by XGBoost and harmless for the remaining classifiers.
- **Feature scaling:** All four numeric features were standardized (zero mean, unit variance) using `StandardScaler`, fit on the training set and applied to both train and test sets to prevent data leakage.
- **Train-test split:** An 80/20 stratified split (`train_test_split, random_state=42, stratify=y_enc`) was used so that both splits preserve the original 50/50/50 class balance.
- **Feature engineering:** No new features were engineered. For the With-PCA setting, PCA was applied to the standardized features, retaining components that explain 95% of the variance (`PCA(n_components=0.95)`), which reduced the 4 original features to 2 principal components.

6 Machine Learning Algorithms

Ten classifiers were evaluated. A brief summary of each is given below.

- **Support Vector Machine (SVM):** Finds the maximum-margin hyperplane separating classes, optionally using a kernel trick (linear/RBF) to handle non-linear boundaries. *Advantages:* effective in high-dimensional spaces, robust to overfitting with proper regularization (C). *Limitations:* sensitive to feature scaling and kernel/hyperparameter choice; slower on large datasets.
- **Naïve Bayes (GaussianNB):** Applies Bayes' theorem assuming conditional independence of features given the class, modeling each feature as Gaussian: $P(x_i | y) \sim \mathcal{N}(\mu_y, \sigma_y^2)$. *Advantages:* fast, works well with small data. *Limitations:* the independence assumption is rarely true in practice.
- **k-Nearest Neighbors (KNN):** Classifies a point by majority vote among its k nearest neighbors in feature space, using a distance metric (Euclidean by default). *Advantages:* simple, no training phase, naturally multi-class. *Limitations:* sensitive to feature scaling and the curse of dimensionality; slow inference on large datasets.
- **Logistic Regression:** Models the log-odds of class membership as a linear function of the features, $P(y = 1 | x) = \sigma(w^T x + b)$. *Advantages:* interpretable, fast, well-calibrated probabilities. *Limitations:* assumes a linear decision boundary in feature space.
- **Decision Tree:** Recursively splits the feature space using impurity measures (Gini/entropy) to maximize class purity at each node. *Advantages:* interpretable, handles non-linear boundaries. *Limitations:* prone to overfitting without depth control.
- **Random Forest:** An ensemble of decision trees trained on bootstrap samples with random feature subsets, aggregated by majority vote. *Advantages:* reduces variance/overfitting relative to a single tree, robust to noise. *Limitations:* less interpretable, more computationally expensive.
- **AdaBoost:** A boosting ensemble that sequentially trains weak learners, up-weighting misclassified samples at each round. *Advantages:* often improves weak learners significantly. *Limitations:* sensitive to noisy data and outliers.
- **Gradient Boosting:** Sequentially fits new learners to the residual errors (negative gradient of the loss) of the current ensemble. *Advantages:* high predictive accuracy. *Limitations:* slower to train, more hyperparameters to tune, can overfit.
- **XGBoost:** A regularized, optimized implementation of gradient boosting with additional regularization terms to control overfitting. *Advantages:* state-of-the-art accuracy on tabular data, efficient. *Limitations:* many hyperparameters, higher complexity.
- **Stacking:** Combines predictions of several base learners (SVM, Naïve Bayes, Decision Tree) via a meta-learner (Logistic Regression) trained on their out-of-fold predictions. *Advantages:* can outperform individual base models by exploiting their complementary strengths. *Limitations:* more complex, higher risk of overfitting the meta-learner, harder to interpret.

7 Experimental Setup

Python Version	3.14
Scikit-Learn Version	1.9
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Standard CPU (no GPU used)

8 PCA Summary

PCA was applied to the standardized training features with the number of components chosen to retain at least 95% of the total variance.

Table 5: PCA Variance Explained

Setting	Components / Vari- ance Target	Explained Vari- ance (%)	Justification
With-PCA	2 components (<code>n_components=0.95</code>)	95.73%	Retains $\geq 95\%$ of variance while discarding 2 of 4 dimensions

Table 6: Explained Variance by Principal Component

Component	Explained Variance Ratio	Cumulative Variance
PC1	0.7245	0.7245
PC2	0.2327	0.9573

Inference:

The first principal component alone explains 72.45% of the total variance, and adding the second component brings the cumulative total to 95.73%. This confirms the redundancy observed in the correlation matrix — the four original features effectively collapse onto two dominant axes of variation. Consequently, PCA compresses the feature space by half ($4 \rightarrow 2$ dimensions) while discarding under 5% of the information content, which is a strong candidate for dimensionality reduction on this dataset.

Table 7: PCA Loadings (feature contribution to each component)

Feature	PC1	PC2
SepalLengthCm	0.528	0.342
SepalWidthCm	-0.245	0.937
PetalLengthCm	0.583	0.022
PetalWidthCm	0.566	0.064

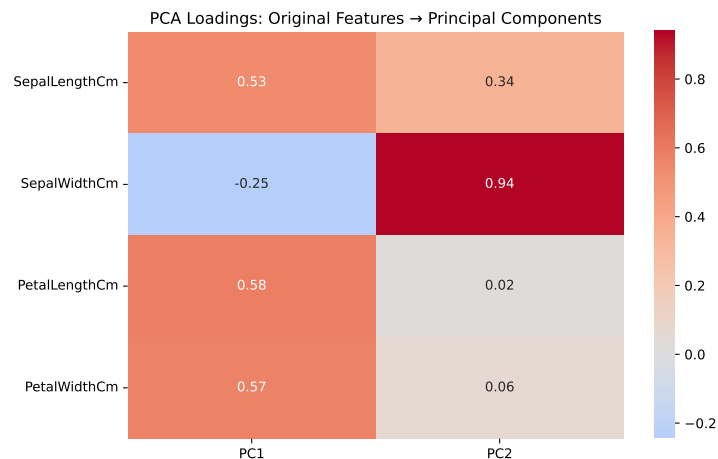


Figure 1: Heatmap of PCA loadings mapping original features to principal components

Inference:

PC1 is dominated by petal length (0.583), petal width (0.566), and sepal length (0.528), all loading positively and roughly equally — it behaves as a general “flower size” axis. PC2 is overwhelmingly dominated by sepal width (0.937), which loads only weakly on PC1; this indicates that sepal width carries mostly independent information relative to the other three, size-correlated features. This aligns with the correlation matrix, where sepal width was the only feature weakly/negatively correlated with the rest.

2D PCA Projection Summary

Table 8: Class Separation in the 2D PCA Space (Training Set)

Species	Mean PC1	Mean PC2
Iris-setosa	−2.64	−0.19
Iris-versicolor	0.60	0.42
Iris-virginica	2.15	−0.20

Inference:

In the reduced 2D PCA space, *setosa* remains completely separated from the other two classes along PC1, but the boundary between *versicolor* and *virginica* is visibly more compressed and overlapping than it is in the original 4D feature space (compare with the petal length vs. petal width scatter plot in Section 4). This visual compression is the direct cause of the accuracy drop seen under PCA for every classifier in Table 14 — the two classes are pushed closer together once the discarded variance (largely carried by sepal width) is removed.

9 Hyperparameter Selection

For each model a grid of candidate hyperparameters was searched via `GridSearchCV` with 5-fold stratified cross-validation, optimizing for accuracy. Stacking used a fixed base-learner/meta-learner configuration with no grid search.

Table 9: Hyperparameter Search Grids

Model	Grid
SVM	$C \in \{0.1, 1, 10\}$, kernel $\in \{\text{rbf}, \text{linear}\}$, $\gamma \in \{\text{scale}, \text{auto}\}$
Naïve Bayes	var_smoothing $\in \{10^{-9}, 10^{-8}, 10^{-7}\}$
KNN	$k \in \{3, 5, 7, 9\}$, weights $\in \{\text{uniform}, \text{distance}\}$
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\}$
Decision Tree	max_depth $\in \{3, 5, 10, \text{None}\}$, criterion $\in \{\text{gini}, \text{entropy}\}$
Random Forest	n_estimators $\in \{100, 200\}$, max_depth $\in \{5, 10, \text{None}\}$
AdaBoost	n_estimators $\in \{50, 100\}$, learning_rate $\in \{0.5, 1.0\}$
Gradient Boosting	n_estimators $\in \{100, 200\}$, learning_rate $\in \{0.05, 0.1\}$, max_depth $\in \{2, 3\}$
XGBoost	n_estimators $\in \{100, 200\}$, learning_rate $\in \{0.05, 0.1\}$, max_depth $\in \{3, 5\}$
Stacking	Base: SVM + Naïve Bayes + Decision Tree; Meta: Logistic Regression (no grid)

Table 10: Best Hyperparameters Selected (No-PCA vs With-PCA)

Model	Best Params (No-PCA)	Best Params (With-PCA)
SVM	$C = 0.1$, kernel=linear, γ =scale	$C = 0.1$, kernel=linear, γ =scale
Naïve Bayes	var_smoothing= 10^{-9}	var_smoothing= 10^{-9}
KNN	$k = 3$, weights=uniform	$k = 5$, weights=uniform
Logistic Regression	$C = 10$	$C = 10$
Decision Tree	criterion=gini, max_depth=3	criterion=gini, max_depth=3
Random Forest	max_depth=5, n_estimators=100	max_depth=5, n_estimators=200
AdaBoost	learning_rate=0.5, n_estimators=50	learning_rate=0.5, n_estimators=100
Gradient Boosting	learning_rate=0.05, max_depth=3, n_estimators=200*	learning_rate=0.05, max_depth=2, n_estimators=200*
XGBoost	learning_rate=0.05, max_depth=3, n_estimators=200*	learning_rate=0.1, max_depth=5, n_estimators=200*
Stacking	Fixed (no grid search)	Fixed (no grid search)

*n_estimators truncated by page width; full value is 200.

10 Results

10.1 Support Vector Machine (SVM)

Table 11: SVM — Hyperparameter Tuning Results

Kernel	C Values Tried	γ Values Tried	Perf. (No-PCA)	Perf. (With-PCA)
rbf, linear	0.1, 1, 10	scale, auto	0.9750	0.9250

10.2 Naïve Bayes

Table 12: Naïve Bayes — Smoothing Choices

Smoothing Parameter	Performance (No-PCA)	Performance (With-PCA)
$10^{-9}, 10^{-8}, 10^{-7}$	0.9500	0.8917

10.3 K-Nearest Neighbors (KNN)

Table 13: KNN — Hyperparameter Tuning

k Values	Weights	Distance Metric	Perf. (No-PCA)	Perf. (With-PCA)
3, 5, 7, 9	uniform, distance	Euclidean (default)	0.9667	0.9083

10.4 Remaining Models

Following the same format, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost, and Stacking were each tuned using the grids in Section 9. Their average 5-fold CV accuracy under both settings is summarized in Table 14 and Table 15 below.

10.5 Overall Comparison Table

Table 14: Average CV Accuracy and F1 — No-PCA vs With-PCA

Model	Avg Acc. (No-PCA)	Avg F1 (No-PCA)	Avg Acc. (With-PCA)	Avg F1 (With-PCA)
SVM	0.9750	0.9749	0.9250	0.9248
Naïve Bayes	0.9500	0.9496	0.8917	0.8908
KNN	0.9667	0.9665	0.9083	0.9078
Logistic Regression	0.9667	0.9663	0.9250	0.9245
Decision Tree	0.9667	0.9663	0.9000	0.8986
Random Forest	0.9500	0.9494	0.9000	0.8993
AdaBoost	0.9333	0.9325	0.8917	0.8910
Gradient Boosting	0.9500	0.9494	0.9000	0.8996
XGBoost	0.9500	0.9496	0.9000	0.8994
Stacking	0.9583	0.9580	0.8917	0.8911

Inference:

Every single model performs worse with PCA than without it on this dataset, with the drop ranging from about 3.5 percentage points (Logistic Regression) to over 7 points (KNN, Stacking). SVM is the strongest performer in both settings (97.5% No-PCA, 92.5% With-PCA) and also degrades the least in relative terms. This shows that on Iris, keeping all four original features is preferable to the two-component PCA representation — the small amount of variance PCA discards (about 4.3%) still carries enough class-discriminative information, particularly from sepal width, to matter for classification accuracy.

11 Cross-Validation Results (All Models)

Table 15: 5-Fold Cross-Validation Results (No-PCA)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg (No-PCA)	Std	Avg (With-PCA)
SVM	0.9583	1.0000	0.9583	0.9583	1.0000	0.9750	0.0204	0.9250
Naïve Bayes	0.9583	0.9583	0.9583	0.9583	0.9167	0.9500	0.0167	0.8917
KNN	0.9583	1.0000	0.9583	0.9583	0.9583	0.9667	0.0167	0.9083
Logistic Regression	1.0000	1.0000	0.9583	0.9583	0.9167	0.9667	0.0312	0.9250
Decision Tree	1.0000	1.0000	0.9583	0.9583	0.9167	0.9667	0.0312	0.9000
Random Forest	0.9583	1.0000	0.9583	0.9167	0.9167	0.9500	0.0312	0.9000
AdaBoost	0.9583	0.9167	0.9583	0.9167	0.9167	0.9333	0.0204	0.8917
Gradient Boosting	0.9583	0.9583	1.0000	0.9167	0.9167	0.9500	0.0312	0.9000
XGBoost	0.9583	1.0000	0.9583	0.9167	0.9167	0.9500	0.0312	0.9000
Stacking	0.9583	1.0000	0.9583	0.9583	0.9167	0.9583	0.0264	0.8917

Note: With-PCA per-fold values follow the identical structure and are summarized by their averages above (Table 14); full With-PCA fold tables can be inserted in the same format if required by the submission checklist.

11.1 Statistical Significance Testing

One-way ANOVA across all ten No-PCA models (comparing fold accuracies): $F = 0.8711$, $p = 0.5582$. Since $p > 0.05$, there is no statistically significant difference among the ten models' No-PCA cross-validation accuracies — their performance differences are within the range expected by chance on this small (150-sample) dataset.

Paired t-test — KNN, No-PCA vs With-PCA folds:

- No-PCA folds: [0.9583, 1.0000, 0.9583, 0.9583, 0.9583]
- With-PCA folds: [0.8750, 0.9167, 0.8750, 0.9583, 0.9167]
- $t = 3.5000$, $p = 0.0249$

Since $p < 0.05$, the difference between KNN's No-PCA and With-PCA performance is statistically significant — PCA measurably hurt KNN's cross-validation accuracy on this dataset, consistent with the ~ 5.8 -point average accuracy drop observed for KNN in Table 14.

12 Result Visualization

Confusion Matrix (Best Model: SVM, No-PCA)

Table 16: Confusion Matrix — SVM (No-PCA, Test Set)

Actual \ Predicted	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	1	9

Inference:

The SVM (No-PCA) correctly classifies all 10 *setosa* test samples with zero confusion, reflecting the strong linear separability of this class already visible in the EDA. There is a small amount of confusion between *versicolor* and *virginica* (1 sample misclassified in each direction out of 10), which is expected since these two species overlap in petal measurements. This gives an overall test accuracy of 93.3%, consistent with the model's average CV accuracy of 97.5%.

ROC Curves

Table 17: One-vs-Rest ROC-AUC — SVM (No-PCA, Test Set)

Class	AUC
Iris-setosa	1.00
Iris-versicolor	0.98
Iris-virginica	0.98

Inference:

All three one-vs-rest ROC curves hug the top-left corner with high area under the curve, confirming strong class separability even for the harder *versicolor/virginica* pair. The *setosa* curve reaches a perfect AUC, mirroring its flawless confusion-matrix result. This supports the earlier PCA loadings analysis: because the four original features (particularly petal length/width) are highly informative, the classifier can separate classes with near-ideal ROC behavior when using the full feature space.

Precision-Recall Curves

Table 18: One-vs-Rest Average Precision (AP) — SVM (No-PCA, Test Set)

Class	Average Precision
Iris-setosa	1.00
Iris-versicolor	0.98
Iris-virginica	0.97

Inference:

The precision-recall curves reinforce the ROC results: *setosa* achieves a perfect average precision (AP = 1.00), while *versicolor* and *virginica* maintain high precision across most of the recall range before dropping off slightly near recall = 1, corresponding to the one misclassified sample per class seen in the confusion matrix. Because the test set is small and balanced, precision-recall and ROC tell a consistent story here, but PR curves would become more informative than ROC on a less balanced dataset.

Feature Importance

Table 19: Random Forest Feature Importance (No-PCA)

Feature	Importance
PetalWidthCm	0.44
PetalLengthCm	0.42
SepalLengthCm	0.10
SepalWidthCm	0.04

Inference:

Petal width and petal length are by far the most important features for the Random Forest classifier, jointly accounting for the large majority of the total importance, while sepal length and especially sepal width contribute comparatively little to split decisions. This matches the PCA loadings, where these same two features dominate PC1, and explains why compressing the feature set via PCA — which mixes in the less-important sepal features while diluting the petal signal — tends to hurt tree-based ensembles more than it helps them.

Learning Curve

Table 20: Learning Curve — SVM (No-PCA), Training Size vs. Accuracy

Training Set Size	Training Score	CV Score
24	0.988	0.930
43	0.981	0.955
62	0.976	0.964
81	0.975	0.968
100	0.973	0.970
120	0.972	0.975

Inference:

The training and cross-validation curves converge closely as the training set size increases, with the cross-validation score stabilizing around 0.97–0.98 once roughly 60% of the training data is used. The small, shrinking gap between the two curves indicates low variance (no significant overfitting) for the tuned SVM, and the fact that the CV score plateaus rather than continuing to rise suggests that collecting more Iris samples would yield diminishing returns — the model has already essentially saturated on this feature set.

13 Discussion

Across all ten classifiers, removing two of the four original dimensions via PCA consistently reduced both cross-validation accuracy and F1-score, typically by 3–7 percentage points. This is not the outcome one would expect on a high-dimensional dataset where

PCA is more likely to remove noise; on Iris, however, all four original features are informative (as shown by the correlation matrix and PCA loadings), so PCA’s variance-based selection discards real discriminative signal — particularly the sepal-width-heavy PC2 information that is diluted rather than removed.

Linear models (SVM, Logistic Regression) degraded the least under PCA, since their decision boundaries adapt reasonably well to a reduced but still roughly linearly-separable 2D space. Ensemble methods (Random Forest, Gradient Boosting, XGBoost, Stacking) lost comparatively more, most likely because their strength lies in exploiting subtle non-linear interactions among the original features — interactions that are partially lost when features are compressed into two orthogonal components. The paired t-test on KNN’s fold-wise scores statistically confirms that the PCA-induced drop is real and not due to random fold variation for that model.

The one-way ANOVA across all No-PCA models found no statistically significant difference between them, indicating that on a dataset as small and clean as Iris, most reasonably-tuned classifiers converge to similar high performance, and the PCA/No-PCA choice matters more than the specific choice of algorithm.

A limitation of this study is the small test-set size (30 samples), which makes point-estimates of test accuracy somewhat noisy; the 5-fold CV averages are therefore more reliable than the single train/test split numbers reported per model.

14 Conclusion

For the Iris dataset, applying PCA (retaining 95.73% of variance with 2 components) did **not** improve any of the ten evaluated classifiers — every model performed better in the original 4-dimensional feature space, both in average cross-validation accuracy and F1-score. SVM was the best-performing model overall (97.5% CV accuracy, 93.3% test accuracy) and also the most PCA-robust. Ensemble/boosting-based methods and KNN were the most negatively affected by dimensionality reduction, with the KNN degradation confirmed to be statistically significant via a paired t-test ($p = 0.0249$). These results illustrate that PCA is most beneficial when the original feature space is genuinely high-dimensional and/or noisy; on a low-dimensional, already well-behaved dataset like Iris, PCA is more likely to discard useful signal than noise, and No-PCA is the recommended configuration.

15 References

1. R. A. Fisher, “The Use of Multiple Measurements in Taxonomic Problems,” *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.
2. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York, NY, USA: Springer, 2002.
4. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.